Министерство науки и высшего образования Российской Федерации

Федеральное государственное бюджетное образовательное учреждение высшего образования

«Владимирский государственный университет имени Александра Григорьевича и Николая Григорьевича Столетовых»

Кафедра информатики и защиты информации

ЛАБОРАТОРНАЯ РАБОТА №2

на тему

Разработка клиент-серверного приложение на основе Socket

Работу выполнил:

студент 1 курса группы ИБ-125

Лисовой Савелий

Работу принял:

Старший преподаватель

Матвеев С.Н.

Владимир 2026

Цель работы

Приобретение практических навыков в области сетевого программирования, реализации многопоточного или асинхронного сервера, проектирования прикладного протокола и организации взаимодействия между процессами.

Задание (Вариант № 20)

Система опроса состояния сервисов. Сервер принимает от клиентов (агентов) периодические сообщения о их состоянии (CPU, RAM, Disk Space). По запросу от администратора выводит сводку. Клиент-Агент собирает метрики своей системы и отправляет их серверу. Клиент-Монитор подключается к серверу и запрашивает красивый отчет о состоянии всех агентов**.**

Ход работы

1.Код серверной части

```
1. import socket
   import json
   import threading
   from datetime import datetime

   class ServiceMonitorServer:
       def __init__(self, host='localhost', port=8888):
           self.host = host
           self.port = port
           self.agents_data = {}
           self.lock = threading.Lock()
           self.running = True
       def handle_client(self, client_socket, address):
           try:
               while self.running:
                   data = client_socket.recv(4096)
                   if not data:
                       break
                   message = json.loads(data.decode('utf-8'))
                   if message['type'] == 'agent_report':
                       self.process_agent_report(message['data'])
                       client_socket.send(json.dumps({'status':
   'ok'}).encode('utf-8'))
                   elif message['type'] == 'monitor_request':
                       report = self.generate_report()
                       client_socket.send(json.dumps(report).encode('utf-
   8'))
           except Exception as e:
               print(f"Ошибка при обработке клиента {address}: {e}")
           finally:
```

```
31                 client_socket.close()
32         def process_agent_report(self, data):
33             with self.lock:
34                 agent_id = data['agent_id']
35                 self.agents_data[agent_id] = {
36                     'cpu': data['cpu'],
37                     'ram': data['ram'],
38                     'disk': data['disk'],
39                     'last_update':
40 datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
41                     'ip': data.get('ip', 'unknown')
42                 }
43                 self.cleanup_old_data()
44         def cleanup_old_data(self):
45             current_time = datetime.now()
46             stale_agents = []
47             for agent_id, data in self.agents_data.items():
48                 last_update = datetime.strptime(data['last_update'],
49 '%Y-%m-%d %H:%M:%S')
50                 if (current_time - last_update).seconds > 300:  # 5 минут
51                     stale_agents.append(agent_id)
52             for agent_id in stale_agents:
53                 del self.agents_data[agent_id]
54         def generate_report(self):
55             with self.lock:
56                 report = {
57                     'timestamp':
58 datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
59                     'total_agents': len(self.agents_data),
60                     'agents': []
61                 }
62                 for agent_id, data in self.agents_data.items():
63                     status = self.determine_status(data)
64                     agent_info = {
65                         'agent_id': agent_id,
66                         'ip': data['ip'],
67                         'cpu': data['cpu'],
68                         'ram': data['ram'],
69                         'disk': data['disk'],
70                         'status': status,
71                         'last_update': data['last_update']
72                     }
73                     report['agents'].append(agent_info)
74                 return report
75         def determine_status(self, data):
76             """Определение статуса агента на основе метрик"""
77             if data['cpu'] > 90:
78                 return 'CRITICAL'
79             elif data['cpu'] > 70:
80                 return 'WARNING'
81             elif data['ram'] > 90:
82                 return 'CRITICAL'
83             elif data['ram'] > 70:
84                 return 'WARNING'
85             elif data['disk'] > 90:
86                 return 'CRITICAL'
87             elif data['disk'] > 70:
88                 return 'WARNING'
89             else:
90                 return 'OK'
91         def start(self):
92             server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
93             server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

```
        server.bind((self.host, self.port))
        server.listen(5)

        print(f"Сервер мониторинга запущен на {self.host}:{self.port}")

        try:
            while self.running:
                client_socket, address = server.accept()
                print(f"Подключен клиент: {address}")
                client_thread = threading.Thread(
                    target=self.handle_client,
                    args=(client_socket, address)
                )
                client_thread.daemon = True
                client_thread.start()
        except KeyboardInterrupt:
            print("\nОстановка сервера...")
        finally:
            self.running = False
            server.close()
if __name__ == "__main__":
    server = ServiceMonitorServer()
    server.start()
```

2.Код Агента клиентской части

```
import socket
import json
import time
import psutil
import platform
import uuid
import argparse


class MonitoringAgent:
    def __init__(self, server_host='localhost', server_port=8888,
agent_id=None):
        self.server_host = server_host
        self.server_port = server_port
        self.agent_id = agent_id or self.get_agent_id()
        self.running = True
    def get_agent_id(self):
        hostname = platform.node()
        return f"{hostname}-{uuid.uuid4().hex[:8]}"
    def collect_metrics(self):
        try:
            cpu_percent = psutil.cpu_percent(interval=1)
            memory = psutil.virtual_memory()
            ram_percent = memory.percent
            disk = psutil.disk_usage('/')
            disk_percent = disk.percent
            return {
                'cpu': round(cpu_percent, 2),
                'ram': round(ram_percent, 2),
                'disk': round(disk_percent, 2)
            }
        except Exception as e:
            print(f"Ошибка при сборе метрик: {e}")
            return None
    def send_report(self, metrics):
        try:
            client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            client.connect((self.server_host, self.server_port))
            message = {
                'type': 'agent_report',
                'data': {
                    'agent_id': self.agent_id,
                    'ip': socket.gethostbyname(socket.gethostname()),
                    **metrics
                }
            }
            client.send(json.dumps(message).encode('utf-8'))
            response = client.recv(1024).decode('utf-8')
            client.close()
            return json.loads(response)
        except Exception as e:
            print(f"Ошибка при отправке отчета: {e}")
            return None
    def run(self, interval=60):
        print(f"Агент мониторинга запущен (ID: {self.agent_id})")
        print(f"Отправка данных на сервер
{self.server_host}:{self.server_port}")
        print(f"Интервал отправки: {interval} секунд")
        try:
            while self.running:
                metrics = self.collect_metrics()
```

```
                if metrics:
                    print(f"[{time.strftime('%Y-%m-%d %H:%M:%S')}] "
                          f"CPU: {metrics['cpu']}%, RAM: {metrics['ram']}%, Disk: {metrics['disk']}%")
                    response = self.send_report(metrics)
                    if response and response.get('status') == 'ok':
                        print("Отчет успешно отправлен")
                    else:
                        print("Ошибка при отправке отчета")
                time.sleep(interval)
        except KeyboardInterrupt:
            print("\nОстановка агента...")
    def stop(self):
        self.running = False
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description='Агент мониторинга')
    parser.add_argument('--host', default='localhost', help='Сервер мониторинга')
    parser.add_argument('--port', type=int, default=8888, help='Порт сервера')
    parser.add_argument('--interval', type=int, default=60, help='Интервал отправки (секунды)')
    args = parser.parse_args()
    agent = MonitoringAgent(args.host, args.port)
    agent.run(args.interval)
```

3.Код администраторской части(Монитор)

```
import socket
import json
import argparse
class MonitoringClient:
    def __init__(self, server_host='localhost', server_port=8888):
        self.server_host = server_host
        self.server_port = server_port
    def request_report(self):
        try:
            client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            client.connect((self.server_host, self.server_port))
            message = {'type': 'monitor_request'}
            client.send(json.dumps(message).encode('utf-8'))
            data = client.recv(65536).decode('utf-8')
            client.close()
            return json.loads(data)
        except Exception as e:
            print(f"Ошибка при запросе отчета: {e}")
            return None
    def format_report(self, report):
        if not report:
            return "Не удалось получить отчет от сервера"
        print("\n" + "=" * 80)
        print(f"ОТЧЕТ О СОСТОЯНИИ СЕРВИСОВ")
        print(f"Время формирования: {report['timestamp']}")
        print(f"Всего агентов: {report['total_agents']}")
        print("=" * 80)
        if report['total_agents'] == 0:
            print("Нет активных агентов")
            return
        print(f"{'Агент':<20} {'IP':<15} {'CPU':<8} {'RAM':<8} {'Disk':<8}
{'Статус':<10} {'Последнее обновление':<20}")
        print("-" * 80)
        for agent in report['agents']:
            status = agent['status']
            if status == 'OK':
                status_display = f"\033[92m{status}\033[0m"  # Зеленый
            elif status == 'WARNING':
                status_display = f"\033[93m{status}\033[0m"  # Желтый
            else:
                status_display = f"\033[91m{status}\033[0m"  # Красный
            print(f"{agent['agent_id']:<20} {agent['ip']:<15} "
                  f"{agent['cpu']:>6}% {agent['ram']:>6}% {agent['disk']:>6}%
"
                  f"{status_display:<10} {agent['last_update']:<20}")
        print("=" * 80)
        status_counts = {'OK': 0, 'WARNING': 0, 'CRITICAL': 0}
        for agent in report['agents']:
            status_counts[agent['status']] += 1
        print(f"\nСтатистика:")
        print(f"OK: {status_counts['OK']} агентов")
        print(f"WARNING: {status_counts['WARNING']} агентов")
        print(f"CRITICAL: {status_counts['CRITICAL']} агентов")
        print(f"\nРекомендации:")
        critical_agents = [a for a in report['agents'] if a['status'] ==
'CRITICAL']
        if critical_agents:
            for agent in critical_agents:
                print(f"Требуется внимание: {agent['agent_id']}")
                if agent['cpu'] > 90:
                    print(f"     - Высокая загрузка CPU: {agent['cpu']}%")
```

```
                if agent['ram'] > 90:
                    print(f"     - Высокое использование RAM:
{agent['ram']}%")
                if agent['disk'] > 90:
                    print(f"     - Мало свободного места на диске: {100 -
agent['disk']}% свободно")
        else:
            print("Все системы работают в штатном режиме")
    def run(self):
        report = self.request_report()
        self.format_report(report)
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description='Клиент мониторинга')
    parser.add_argument('--host', default='localhost', help='Сервер
мониторинга')
    parser.add_argument('--port', type=int, default=8888, help='Порт
сервера')

    args = parser.parse_args()

    client = MonitoringClient(args.host, args.port)
    client.run()
```

4.Скриншоты демонстрации работы клиента и сервера

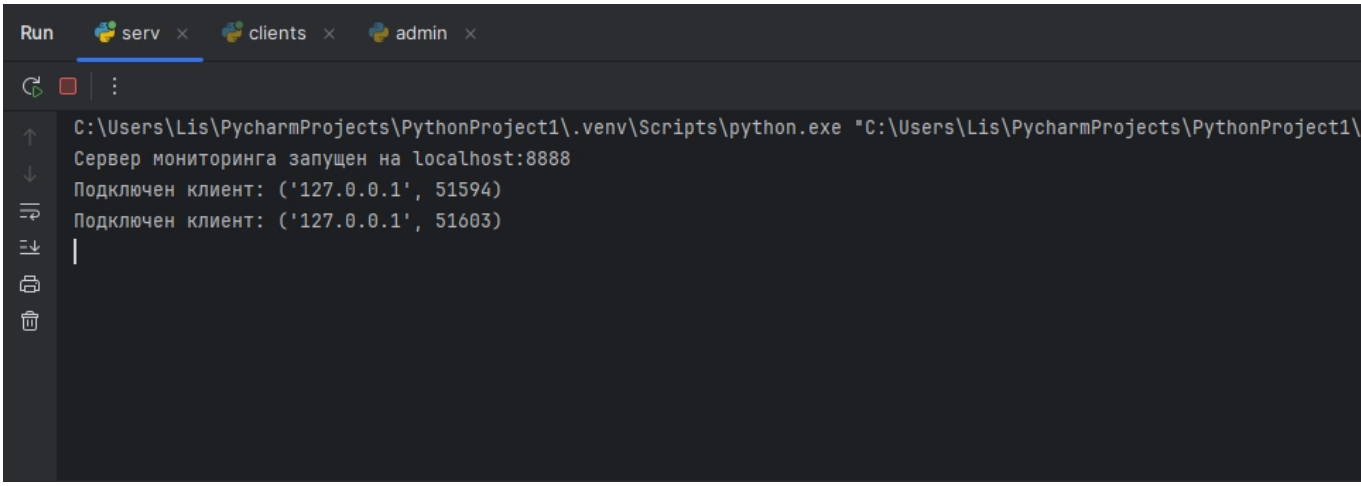

Рисунок 1— Запуск сервера и подключение клиентов

Рисунок 2— Отправка данных на сервер от клиента

```
Run    serv ×    clients ×    admin ×

C:\Users\Lis\PycharmProjects\PythonProject1\.venv\Scripts\python.exe "C:\Users\Lis\PycharmProjects\PythonP

================================================================================
ОТЧЕТ О СОСТОЯНИИ СЕРВИСОВ
Время формирования: 2026-03-26 16:59:02
Всего агентов: 1
================================================================================
Агент                IP              CPU      RAM      Disk     Статус     Последнее обновление
--------------------------------------------------------------------------------
Lis-b478f1f9         192.168.0.101     20.4%   53.3%   56.6% OK 2026-03-26 16:58:56
================================================================================
```

Рисунок 3— Получение данных администратором от сервера

5.Ссылка на код, размещенный в публичном Github репозитории:

https://github.com/KitsuneSempai/Labarotornya-rabota-2